

CableDoc Desktop

De la patchera de papel al grafo dirigido

Un review exhaustivo, banco por banco, de una herramienta de documentación de cableado de planta que dejó de ser una planilla con dibujitos y pasó a razonar sobre su propia topología

Por un ingeniero de planta con paso por desarrollo de software • Edición técnica, Agosto 2026

Hay una escena que cualquiera que haya pasado guardias en un máster de aire conoce de memoria: alguien en control llama porque “se fue la señal de TELEFE SAT”, y el técnico de planta baja al rack con una carpeta de fichas, un rotulador y la esperanza de que la última persona que tocó esa patchera haya anotado el cambio. En el 90% de los casos, no lo hizo. CableDoc Desktop nace exactamente de esa bronca: documentar cableado no como un archivo muerto sino como un modelo vivo que se puede interrogar — “si corto este cable, ¿qué se queda sin aire?” — y que responda en milisegundos, no en media hora de seguir cables a mano por el rack.

Tuve acceso al código fuente completo del proyecto tal como está hoy: 89 archivos, unas 63.000 líneas contando SQL y documentación, de las cuales aproximadamente 32.000 son Python puro repartidas en 19 módulos. Lo que sigue es un review de ingeniería, no de marketing: qué decisiones de arquitectura sostienen la aplicación, dónde se nota la cicatriz de una reescritura completa desde una versión anterior en VB.NET, qué patrones se repiten con disciplina y — porque ningún sistema real está libre de esto — qué bugs de producción encontré documentados con una honestidad que pocas bitácoras de desarrollo tienen.

En una frase: CableDoc pasó de “dibujar el cableado” a “simular el cableado”. Ese salto --- de un modelo de datos pasivo a un motor de grafos que puede responder preguntas de impacto y riesgo --- es el hilo conductor de todo este review.

1. El punto de partida: qué es CableDoc

CableDoc Desktop es una aplicación de escritorio en Python 3 sobre GTK3 (PyGObject), con SQLite como

única base de datos — sin servidor, sin dependencias externas de infraestructura — pensada para documentar y analizar el cableado y el flujo de señal de una planta de broadcast/AV: equipos, conectores, cables, patcheras, matrices de ruteo, racks, salas, y — desde la incorporación más reciente y ambiciosa — la señal misma como entidad de primera clase: qué contenido circula por cada conector, de dónde sale y a dónde llega.

Ficha técnica

- **Stack:** Python 3.10+, GTK3/PyGObject, Cairo, SQLite 3
- **Motor de grafos:** GraphQLite (`graphqlite.graph.Graph`), extensión de SQLite en Rust/C con soporte de Cypher
- **Esquema:** 44 tablas, 14 vistas SQL
- **Código Python:** ~32.000 líneas en 19 módulos
- **i18n:** español (base) + inglés + portugués, 334 claves traducidas
- **Datos de producción relevados:** ~414 equipos, 216 patcheras, 35 equipos tipo FANTASMA

Lo que hace interesante a este proyecto para una nota técnica no es la lista de funcionalidades — que es larga y sigue creciendo — sino que es un caso de estudio poco común: una aplicación de dominio muy específico (cableado de planta de TV/radio) desarrollada por alguien con el perfil exacto para no cometer los errores de modelado que comete el software genérico de gestión de activos cuando intenta “adivinar” cómo funciona una patchera física. Eso se nota en el schema, y se nota todavía más en el motor de análisis de impacto.

2. La herencia: qué sobrevivió de la versión VB.NET

Antes de este CableDoc existió una versión en VB.NET/WinForms. La reescritura completa a Python/GTK3 no fue un simple “port”: cambió el paradigma de renderizado de punta a punta. La versión anterior generaba texto intermedio en formato Graphviz (.dot), invocaba un binario externo dot.exe para rasterizarlo, y en algunos flujos hasta volvía a parsear la salida de ese .dot generado para extraer coordenadas. Es un patrón de integración razonable para su época, pero trae consigo una dependencia binaria externa, un paso de generación de texto que puede desincronizarse del modelo, y una vuelta de parseo que es puro overhead conceptual: generar texto para después releerlo.

La versión Python eliminó Graphviz por completo. El propio README.md del proyecto lo dice sin vueltas: “*Ya NO hace falta Graphviz / el comando dot — los diagramas se dibujan con Cairo, no se generan más PDFs vía Graphviz*”. Todo el dibujo — nodos, curvas Bézier para los patchcords, minimapa, badges de estado, coloreado por señal o por riesgo — se hace directamente sobre Gtk.DrawingArea con Cairo. No hay texto intermedio, no hay proceso externo, no hay re-parseo.

Pero el cambio más profundo no está en el renderizado sino en el análisis. La versión VB.NET no tenía — por lo que puedo reconstruir del alcance del dominio — un motor de simulación de impacto real; el grafo, cuando existía, era principalmente para dibujar. La versión Python incorporó graph_impact.py, que construye el grafo de señal completo en memoria usando **GraphQLite**, una extensión de SQLite escrita en Rust y C con soporte del lenguaje de consulta Cypher. Esto es una decisión de arquitectura poco común y, a mi criterio, acertada: en vez de reimplementar a mano un BFS/Dijkstra sobre diccionarios de Python — lo cual funciona, pero escala mal y es fácil de tener sutilmente mal — se apoya en un motor de grafos real, con su propio lenguaje de consulta, corriendo embebido sin servidor aparte.

Lo único que sobrevivió intacto

De toda la capa de acceso a datos de la versión anterior, hay una sola pieza que cruzó la reescritura completa sin cambios de fondo: la vista SQL CONEXIONES_AMBOS_EXTREMOS, un self-join sobre la vista base CONEXIONES que resuelve, para cada cable, sus dos extremos (equipo, tipo, conector) en

una sola fila. Es la vista que once años de UI — ABMs, exportaciones, el propio motor de grafos vía _leer_bd() — siguen consultando por posición de columna. Que una sola pieza de lógica sobreviva intacta a una reescritura completa de framework, lenguaje de renderizado y motor de análisis dice algo sobre lo bien resuelta que estaba esa consulta desde el principio.

3. Anatomía del código: dónde vive la complejidad

Un review honesto no puede evitar mirar la distribución de tamaño de los archivos, porque ahí es donde se ve — sin necesidad de leer una sola línea — dónde está concentrado el riesgo de mantenimiento.

Los cinco módulos más grandes

- pantallas_avanzadas.py — 9.830 líneas (diagrama de conexiones, patcheras, árbol de racks, todos los mixins de UI avanzada)
- cabledoc.py — 8.211 líneas (ventana principal, menús, todos los diálogos ABM)
- modelo.py — 4.865 líneas (capa de acceso a datos completa)
- cypher_console.py — 1.864 líneas (consola Cypher interactiva)
- graph_impact.py — 1.369 líneas (motor de grafo de señal)

Casi 10.000 líneas en un solo archivo de pantallas (pantallas_avanzadas.py) es, en cualquier proyecto, una señal amarilla de refactor pendiente. Pero mirando el changelog se entiende por qué no es un archivo desordenado sino un archivo *acumulativo por diseño*: el patrón que usa el proyecto para agregar funcionalidad al diagrama principal es el de mixins por herencia múltiple — ImpactoMixin, RiesgoDiagramaMixin, SenalDiagramaMixin, EscenarioMixin, DiagnosticoMixin — cada uno viviendo en su propio archivo (impacto_ui.py, riesgo_diagrama_ui.py, etc.) e inyectando sus propios puntos de integración (_on_press, _on_motion, _on_draw) en la clase DiagramaConexiones de pantallas_avanzadas.py. Es decir: la complejidad de cada subsistema está bien aislada en su

propio módulo de 200 a 750 líneas; lo que crece en `pantallas_avanzadas.py` es la clase anfitriona que los orquesta, no la lógica de negocio de cada uno. Es un patrón de composición razonable para GTK3, donde no existe un sistema de plugins/slots declarativo como en frameworks web modernos.

3.1 El modelo de datos: 44 tablas con separación de catálogo

El esquema SQLite tiene 44 tablas y 14 vistas. Un detalle de diseño que aparece una y otra vez en el changelog, y que vale la pena destacar porque es fácil de pasar por alto en proyectos más chicos: la separación estricta entre tablas de *catálogo* (plantillas reutilizables: `equipo_catalogo`, `conector_catalogo`, `slot_catalogo`, `frame_catalogo`) y tablas de *instancia real* (`equipo`, `conector`, `slot`, `frame`). Cuando se agregó el motor de reglas lógicas AND/OR para compuertas de equipo (pensado para modelar un DSK o cualquier combinador con múltiples entradas condicionales), el proyecto no mezcló la definición de la regla con su instancia: creó `regla_logica_catalogo` / `regla_logica_catalogo_miembro` / `regla_logica_catalogo_salida` como tablas separadas de `regla_logica` / `regla_logica_miembro` / `regla_logica_salida`, replicando a propósito el mismo patrón que ya usaban equipos y conectores. Es el tipo de disciplina que se pierde fácil bajo presión de entrega, y que acá se sostuvo.

4. El corazón del sistema: el motor de impacto de señal

Si tuviera que señalar una sola pieza de código como la más valiosa de todo el proyecto, sería `graph_impact.py` y su clase `GraphImpactAnalyzer`. No por su tamaño (1.369 líneas, modesto comparado con los archivos de UI) sino por lo que resuelve: transformar una topología física de cableado — con toda su ambigüedad de patcheras en bypass, matrices de ruteo configuradas o no, y compuertas lógicas — en un grafo dirigido consultable, y sobre ese grafo responder tres preguntas operativas reales:

- `simular_desconexion(cable_id)` — ¿qué se queda sin señal si corto este cable?
- `simular_falla_equipo(equipo_id)` — ¿qué se queda sin señal si este equipo falla por completo?

- `simular_escenario(cables_cortados, equipos_fallados, conexiones_virtuales)` — la generalización de las dos anteriores a una combinación arbitraria de cambios, evaluada en un solo cálculo, pensada para planificar una reconexión de contingencia antes de tocar un solo cable real.

Lo que hace sofisticado al modelo no es el algoritmo de alcanzabilidad en sí — es, en esencia, un problema de nodos alcanzables desde un conjunto de semillas sobre un grafo con aristas excluidas — sino la fidelidad con la que traduce comportamiento físico real a estructura de grafo. Tres ejemplos concretos, tomados directamente de la bitácora de desarrollo:

Direccionalidad sin texto libre. Cada conector tiene una columna `direccion` (IN/OUT) que hoy orienta cada arista del grafo. No siempre fue así: en una fase anterior documentada en el changelog, la dirección se infería buscando la subcadena "OUT" dentro del *nombre* del tipo de conector, y el filtrado de fuentes de referencia dependía de que el tipo de conector se llamara literalmente REFOUT. Ese acoplamiento a convención de nombreo en español — documentado con una medición real: 31 de 216 patcheras (14,4%) con nombreo atípico que rompía la inferencia — fue identificado, planificado y migrado a columnas de control dedicadas (`tipo_conector.direccion`, `tipo_conector.es_referencia_generada`) de forma explícita, con migraciones idempotentes y sin perder retrocompatibilidad. Es exactamente el tipo de deuda técnica que en la mayoría de los proyectos de este tamaño se queda “para después” porque funciona en el 85% de los casos.

Patcheras como sub-grafo, no como nodo único. Modelar una patchera fue, según el propio changelog, la fuente de al menos dos bugs de producción reales que vale la pena diseccionar porque son un buen ejemplo de cómo un modelo “razonablemente correcto” puede fallar en un caso de borde con consecuencias serias en un análisis de impacto.

Bug real #1 --- la patchera que ocultaba impacto

Un módulo de patchera se modelaba originalmente como un único nodo “pasa todo” en el grafo de señal. Sus cuatro puertos físicos (entrada de atrás, salida de atrás, derivación de adelante, inserción de adelante) compartían el mismo nodo lógico. Consecuen-

cia: cortar el cable que llega al puerto de entrada no aislaba lo que sale por los otros tres puertos si *cualquier* otro cable seguía tocando ese equipo por cualquier conector — el análisis de impacto subestimaba el daño real aguas abajo. La corrección modeló cada uno de los cuatro puertos como su propio nodo (PP:<id_conector>) con las aristas internas del bypass físico real: sin nada patcheado adelante, entrada→salida directo; si se usa la derivación y/o la inserción, ese bypass se interrumpe y la señal sigue por el frente. Validado contra un caso real de la base: antes del fix, cortar un cable específico daba `hay_impacto=False`; después, detecta correctamente 45 equipos y 55 cables impactados aguas abajo.

Bug real #2 --- la regla que perdía contra datos viejos

El segundo bug es más sutil y, para mí, más interesante desde el punto de vista de ingeniería: un equipo con una regla lógica propia (AND/OR, para modelar por ejemplo un DSK con múltiples entradas condicionales) podía tener *además* filas residuales en la tabla de ruteo de matriz para ese mismo equipo — datos viejos de una configuración cargada antes de definir la regla, o de un cambio de rol posterior. Ambos caminos competían por el mismo conector de entrada, y el ruteo de matriz “ganaba” en silencio, dejando el nodo-compuerta de la regla con un miembro que en la práctica ningún cable alimentaba. El síntoma en producción: el análisis de impacto no detectaba *ningún* corte que cambiara algo aguas abajo de ese equipo, porque ya figuraba sin señal de antes — un falso negativo perfectamente silencioso. La corrección fue dar prioridad explícita a la regla propia del equipo sobre cualquier fila residual de matriz, y se confirmó que sólo 1 de todos los equipos de la base real tenía este conflicto. Vale la pena remarcar el método de diagnóstico: no fue una revisión de código, fue un reporte de usuario (“esto no puede ser que no tenga impacto”) seguido de una investigación dirigida por datos reales hasta encontrar la causa exacta.

Los dos bugs comparten algo que dice bien del proceso de desarrollo: ninguno se cerró con un “arreglado, ya anda”. Cada uno tiene su corrección validada contra una copia de la base de datos real — nunca el archivo

original — con conteos concretos de antes/después, documentados en el changelog con el mismo nivel de detalle que tendría un post-mortem de incidente en un equipo de SRE.

4.1 Riesgo como función explícita, no como intuición

Sobre el mismo motor de grafo se construyó `risk_engine.py`, que calcula un Índice de Riesgo de Falla (IRF) por equipo con una fórmula deliberadamente simple y auditable:

```
Riesgo = max(Probabilidad, P_MIN) x Impacto / 100
```

Probabilidad combina antigüedad del equipo, condición de uso e historial de problemas registrados; Impacto reutiliza directamente `GraphImpactAnalyzer.simular_falla_equipo` para calcular qué fracción del parque queda sin señal si ese equipo falla completo. Un detalle de producto que me parece bien pensado: si existe un conjunto de “equipos críticos” marcado a mano — por ejemplo la cadena de aire completa, marcada por selección múltiple en el propio diagrama — el Impacto se mide sólo contra ese conjunto, no contra todo el parque. La diferencia práctica es enorme: sin ese ajuste, un switcher máster y un monitor de sala de control secundaria pesan lo mismo en el cálculo de impacto si ambos afectan, en términos relativos, a una fracción similar del total de equipos. Con el conjunto de críticos activo, el riesgo prioriza lo que realmente importa. Y si la tabla de críticos está vacía, el sistema no rompe: cae de forma explícita al criterio anterior (fracción de todo el parque), documentado como comportamiento esperado, no como caso no contemplado.

5. El asistente de diagnóstico: bisección aplicada a un problema de planta

Uno de los agregados más recientes — y, a mi juicio, uno de los más elegantes desde el punto de vista de diseño de algoritmo aplicado a un problema de dominio muy concreto — es `diagnostico_falla.py`. La idea es simple de enunciar y no trivial de implementar bien: dado un síntoma (“no llega señal acá”), en vez de que el técnico siga el cable físicamente tramo por tramo, el sistema reconstruye la cadena completa de conectores entre el punto de síntoma y la fuente — reutilizando el

grafo invertido que ya construye `senal_visual.py` — y conduce una **bisección real** sobre esa cadena: pregunta por un punto intermedio, según la respuesta descarta la mitad correspondiente, y repite. Es búsqueda binaria aplicada a topología de cableado físico, con la complejidad adicional de que “la mitad” no es un índice de arreglo sino un punto de test físicamente accesible (`conector.es_punto_test`), poblado automáticamente para los jacks frontales de patchera y editable a mano para cualquier otro punto.

Cada corte que no permite reconstruir la cadena completa se clasifica en una de seis categorías explícitas — FUENTE, MATRIZ_SIN_RUTEO, PATCHERA_INCOMPLETA, BIFURCACION, SIN_ORIGEN, BUCLE — en vez de un genérico “no se pudo determinar”. Esa clasificación no es cosmética: le dice al técnico *qué tipo* de problema está mirando (¿la fuente realmente no tiene nada aguas arriba, o es que la matriz simplemente no tiene ruteo cargado para ese camino?), que es información accionable distinta en cada caso.

El caso de borde que más me gustó del changelog: bifurcaciones reales — un equipo con más de una entrada activa, como un embedder de audio/video — se resuelven preguntando explícitamente cuál de las dos ramas corresponde al síntoma *antes* de arrancar la bisección, en vez de intentar adivinar. Y el ajuste posterior a la primera ronda de pruebas end-to-end fue igual de concreto: los jacks frontales de una patchera en bypass total (el caso más común en una planta real: nada patcheado) no aparecían como nodos propios de la cadena, porque desde el punto de vista lógico son el mismo punto que su back correspondiente — se corrigió tratándolos como acceso físico alternativo al mismo nodo lógico, que es exactamente cómo un técnico los usa en la práctica.

6. Internacionalización: conviene automatizar y cuándo no

CableDoc nació en español y se internacionalizó a inglés y portugués sin tocar el modelo de datos ni reescribir la UI. El mecanismo es deliberadamente simple: un diccionario `i18n.py` de 334 claves (clave = texto en español, valor = traducciones por idioma) consultado a través de una función `_()` de estilo `gettext`. Lo que hace interesante al proceso, más que el resultado, es el pipeline de aplicación: en vez de traducir manualmente cientos de literales de string dispersos en 8.000+ líneas de diálogos

GTK, se construyó `auto_wrap.py`, un script basado en el AST de Python que envuelve automáticamente los literales de string relevantes en llamadas a `_()`, operando por *offset de byte* para no correr el riesgo de un reemplazo de texto ingenuo que rompa strings similares en contextos distintos.

Ese enfoque automatizado cubre el caso general, pero — como con cualquier transformación de AST sobre un código de UI real — deja afuera casos que requieren juicio: f-strings con interpolación, literales dentro de tuplas, el texto de radio buttons armado dinámicamente, y constantes evaluadas en tiempo de import (el caso puntual documentado es SNIPPETS, una constante de módulo que tuvo que convertirse en una función `_construir_snippets()` para que la traducción se resolviera en el momento correcto y no en el momento de carga del módulo, antes de que el idioma esté configurado). Es una lección de ingeniería que vale para cualquier pipeline de automatización de refactor: automatizar el 90% mecánico y tratar el 10% restante como una lista de casos especiales conocidos, en vez de forzar el script a cubrir el 100% a costa de reglas cada vez más frágiles.

7. Patrones que se repiten (para bien)

Después de leer changelog y estructura de módulos con cierto detalle, hay un puñado de convenciones que aparecen una y otra vez, entrega tras entrega, y que — para cualquiera que haya mantenido un proyecto de vida larga — son la diferencia entre una base de código que se puede seguir extendiendo con confianza y una que empieza a dar miedo tocar.

Convenciones sostenidas en el tiempo

- **Migraciones idempotentes con sentinel explícito.** Cada `asegurar_tablas_*`() / `asegurar_columnas_*`() en `modelo.py` chequea si la columna/tabla ya existe antes de poblarla, y las migraciones de *datos* (no de esquema) usan una tabla de control dedicada (`_migracion_hardcode_idioma`) como sentinel de “ya corrida”, porque el criterio “la columna no existe” no aplica cuando no hay columna nueva de por medio.
- **Nunca pisar un valor local en import/ex-**

port. El importador de catálogo JSON no sobreescribe silenciosamente un valor que ya existe distinto en destino — devuelve una lista de conflictos explícita, y un diálogo dedicado (`_DialogoConflictosImportacion`) deja que el usuario decida fila por fila, con “mantener lo local” como default.

- **Validación contra copia de la base real, nunca el archivo original.** Es la frase que más se repite en el changelog, casi como una fórmula ritual, y es la práctica correcta: cada migración y cada fix de bug se corre primero contra una copia.
- **Batch en vez de N+1.** Cuando se detecta un patrón de una consulta por nodo en un loop de render (`tiene_regla_logica_efectiva` llamado por nodo), se reemplaza por una consulta batch (`equipos_con_regla_logica_activa()`) — una medición real documentada baja el tiempo de carga de ~2,2 segundos a ~8 milisegundos.

Por qué importa: en un proyecto de un solo desarrollador de largo aliento, la disciplina de proceso --- validar contra datos reales, documentar cada decisión con su motivo, no pisar nunca un valor local sin preguntar --- termina siendo más determinante para la salud del código a largo plazo que cualquier elección puntual de arquitectura.

8. Zonas de fricción y deuda técnica visible

Ningún review serio es sólo elogio. Hay puntos concretos que, si yo estuviera revisando este código antes de un sprint largo de nuevas features, marcaría para atención:

Concentración en dos archivos. `cabledoc.py` (8.211 líneas) y `pantallas_avanzadas.py` (9.830 líneas) concentran juntos más de la mitad del código Python del proyecto. El patrón de mixins mantiene aislada la lógica nueva, pero la clase `DiagramaConexiones` y `VentanaPrincipal` siguen creciendo como puntos de integración obligados. No es un problema urgente — GTK3 no deja muchas alternativas mejores sin introducir un framework de plugins propio — pero es el tipo de archivo que, en algún momento, empieza a costar más abrir en el editor que entender.

Índices numéricos sobre una vista ancha. La decisión documentada de *no* agregar columnas nuevas a `CONEXIONES_AMBOS_EXTREMOS` para no tocar el orden de sus columnas — porque varios call-sites acceden por posición numérica, no por nombre — es pragmática a corto plazo y una fuente de fragilidad a largo plazo. Cualquier futura modificación de esa vista requiere auditar cada consumidor por posición, en vez de que el propio lenguaje (columnas nombradas, o un mapeo explícito a dataclass) haga esa garantía por diseño.

Fallbacks de compatibilidad acumulados. Varias migraciones de “texto libre” a “columna de control” — dirección de conector, rol de patchera, función de puerto — se hicieron con un OR de compatibilidad hacia el criterio viejo (nombre de tipo de conector, prefijo de nombre) en vez de un corte limpio. Es la decisión correcta en el momento (no romper instalaciones existentes con datos no migrados), pero cada fallback que se agrega es una rama de código que alguien tiene que seguir entendiendo. Vale destacar que el propio proyecto fue *deshaciendo* varios de estos fallbacks a medida que confirmaba que ya no hacían falta — la Fase C del plan de función de patchera eliminó explícitamente los fallbacks a prefijo de nombre una vez que `id_funcion_patchera` quedó bien poblado — lo cual es exactamente la disciplina correcta: introducir el fallback, y programar activamente su remoción, en vez de dejarlo indefinidamente “por las dudas”.

Cobertura de pruebas dependiente de entorno gráfico. La validación de UI depende sistemáticamente de GTK real corriendo bajo Xvfb, lo cual es válido y mejor que no probar nada, pero deja a la lógica de negocio pura (`graph_impact.py`, `risk_engine.py`, `escenario_engine.py`) sin una suite de tests automatizados independiente de la disponibilidad de un entorno gráfico y de `graphqlite` instalado — el propio changelog documenta al menos una sesión donde no se pudo instanciar `GraphImpactAnalyzer` end-to-end en el sandbox de prueba y hubo que compensar replicando las queries SQL a mano.

9. Modo Escenario: planificar antes de tocar un cable

Merece su propia sección porque es, de las features más recientes, la que más directamente traduce el motor de grafo en una herramienta operativa de guardia. `escenario_engine.py` (341 líneas, deliberadamente sin ninguna dependencia de GTK) orquesta el modelo y

el analizador de grafo para permitir armar, en memoria, una combinación de cambios — equipos fallados, cables cortados, reconexiones virtuales propuestas — y evaluar su efecto completo con `simular_escenario()` antes de escribir una sola fila en la base real.

El detalle de diseño que más valoro acá es la distinción explícita entre lo que *se puede simular* y lo que *se puede aplicar*: las fallas de equipo son sólo para la simulación — no existe, ni debería inventarse artificialmente, un campo “apagado” en el modelo de datos actual de un equipo real — así que `resumen_aplicar()` y `aplicar_a_infraestructura()` las ignoran a propósito y sólo materializan las reconexiones de cable, dentro de una única transacción explícita. Y antes de aplicar nada, `resumen_aplicar()` arma — sin tocar la base — exactamente qué se va a borrar y qué se va a insertar, para mostrarlo en un diálogo de confirmación. Es la diferencia entre una herramienta que “hace cosas” y una que te deja verificar el plan antes de ejecutarlo, que en un contexto de guardia de aire en vivo no es un lujo de UX sino una salvaguarda operativa real.

Un bug de este mismo subsistema, documentado con la misma honestidad que los anteriores: cuando una reconexión virtual del escenario conectaba el mismo par de equipos que un cable recién cortado — el caso típico de “reconexión de emergencia”, reemplazar un cable por otro entre los mismos dos puntos — la exclusión de aristas cortadas se aplicaba también sobre la arista virtual nueva, porque ambas resolvían al mismo par de nodos a nivel equipo. El fix separó limpiamente la propagación: los cortes sólo se evalúan sobre las aristas reales del grafo, nunca sobre las aristas virtuales del escenario. Es un bug de una línea conceptual, con un impacto operativo grande: sin el fix, la herramienta de contingencia fallaba justo en el escenario de contingencia más común.

10. El grafo invertido: un mismo truco, tres problemas distintos

Hay una decisión de diseño transversal que aparece en al menos tres módulos independientes del proyecto — `senal_propagation.py`, `senal_visual.py` y, en su forma de bisección, `diagnostico_falla.py` — y que merece destacarse aparte porque es la clase de reutilización conceptual que distingue a un desarrollador con criterio de arquitectura de uno que simplemente agrega código nuevo por feature. Los tres módulos resuelven preguntas superficialmente distintas: “¿qué nombre de

señal circula por este cable?” (propagación), “¿qué imagen de referencia visual corresponde a este conector de salida ahora mismo?” (vista previa visual) y “¿en qué tramo está el corte?” (diagnóstico). Pero las tres son, en el fondo, el mismo problema: dado un conector, seguir hacia atrás quién lo alimenta, atravesando los mismos tipos de equipo passthrough (DISTRIBUIDOR, ENRUTADOR, MODULO PATCHERA) con la misma semántica de bypass.

En vez de triplicar esa lógica — o, peor, mantenerla ligeramente distinta en cada módulo por descuido — `senal_visual.py` arma explícitamente un grafo invertido `REV[conector] = el conector que lo alimenta`, con el mismo criterio ya validado en `senal_propagation.py`, y lo describe en su propio docstring como “el mismo truco de grafo invertido que ya usa el resto del proyecto”. La resolución de imagen para cualquier conector queda entonces como una única función recursiva de cuatro pasos — estrategia visual propia o heredada, imagen manual, recursión sobre quien lo alimenta, o sin imagen con motivo explícito — sin necesidad de una rama de código por cada rol_senal posible, porque el grafo invertido ya sólo contiene aristas para los roles que efectivamente son passthrough. Es un ejemplo limpio de cómo modelar bien el problema una vez (¿quién alimenta a quién?) permite resolver preguntas de negocio aparentemente distintas con la misma estructura de datos.

11. La consola Cypher: exponer el motor, no sólo consumirlo

Otra decisión que dice bastante sobre la filosofía del proyecto es `cypher_console.py` (1.864 líneas): un diálogo GTK3 que expone una consola interactiva de Cypher — el lenguaje de consulta que usa GraphQLite — directamente al usuario final, reutilizando el mismo grafo que ya construye `GraphImpactAnalyzer`. No es una herramienta de debugging interna dejada accidentalmente accesible: tiene guardado/carga de archivos `.cypher`, directorio de logs propio, y está pensada como una funcionalidad de producto para un usuario avanzado que quiera hacer preguntas ad-hoc sobre la topología que el resto de la UI no anticipó (“¿qué equipos tienen más de tres cables OUT sin ningún IN?”, por ejemplo, expresado directamente en Cypher en vez de esperar a que alguien programe un reporte específico).

Es una apuesta de producto poco común en software de nicho: en vez de blindar el motor de grafos detrás de

una serie fija de reportes predefinidos, se le da al usuario más avanzado — probablemente el propio desarrollador, pero potencialmente cualquier técnico de planta con curiosidad — una vía directa al mismo lenguaje de consulta que usa el motor internamente. El riesgo obvio (una consulta mal escrita, un usuario que no conoce Cypher y se frustra) se mitiga con lo esperable: manejo de errores explícito, y el hecho de que es una herramienta opcional que convive con — no reemplaza a — los diálogos de impacto y riesgo pensados para el técnico de guardia sin tiempo ni ganas de aprender un lenguaje de consulta a las tres de la mañana.

12. Lienzo libre y vista global de patcheras: UI que sigue al modelo

Dos features de interfaz merecen mención porque muestran cómo la solidez del modelo de datos subyacente termina habilitando funcionalidad de UI que sería mucho más costosa de construir sobre un modelo más débil.

La primera es `diagrama_personalizado.py` (491 líneas): un lienzo libre donde el usuario arma sus propios diagramas ad-hoc — trayendo equipos puntuales, conectando puertos a mano con arrastre y línea punteada de vista previa — y los guarda como entidad propia (`diagrama_guardado` / `_nodo` / `_conexion`), independiente del diagrama general de toda la planta. Vale la pena señalar un episodio de integración documentado en el changelog: el módulo existía en el árbol de archivos pero estaba huérfano — sin import, sin tablas, sin entrada de menú, código muerto e inalcanzable desde la UI — probablemente resultado de trabajo en una rama paralela que nunca se terminó de fusionar. La reintegración se hizo comparando con esa rama paralela, tomando el CRUD completo de un lado pero conservando deliberadamente la interacción de conexión más nueva (arrastre con vista previa) del otro por ser superior, documentando explícitamente esa decisión de merge selectivo en vez de tomar un lado completo a ciegas.

La segunda es la vista global de patcheras en `PatcherasVista`, que pasó de mostrar una patchera a la vez (requiriendo seleccionar un equipo primero) a un modo global — `id_equipo=None` — que muestra todas las patcheras del sistema simultáneamente, con codificación de color por equipo, renderizado de patchcords con curvas Bézier, tooltips unificados, diálogos de doble clic, exportación CSV y un overlay de toggle para re-

saltar cables que cruzan de un rack a otro. Es exactamente el tipo de vista que un técnico de planta real necesita cuando está buscando un patchcord específico y no tiene forma de saber de antemano en cuál de doscientas dieciséis patcheras está — y es una vista que sólo es viable de construir con razonable esfuerzo porque el modelo ya distingue con columnas de control (no con texto libre) qué rol cumple cada equipo y qué función cumple cada puerto.

13. Import/export de catálogo: versionado de formato tomado en serio

Un detalle que revela madurez de producto, no sólo de código: el formato JSON de exportación de catálogo de equipos lleva un número de versión explícito que se fue incrementando — documentado hasta `version: 5` en el changelog — cada vez que se agregó un campo nuevo al catálogo exportado (primero el nombre del tipo de equipo solo, después con `rol_senal`, después con dirección y flag de referencia por conector, finalmente con la función de patchera por clave estable en vez de por id numérico interno).

Lo que hace correcto ese versionado no es el número en sí sino cómo se usa: el importador mantiene compatibilidad hacia atrás explícita con cada versión anterior — un export viejo donde el tipo de equipo era sólo un string plano, sin `rol_senal`, cae a un valor por defecto documentado (DISTRIBUIDOR) en vez de romper el import. Y — ya mencionado en la sección de convenciones, pero vale repetirlo en este contexto puntual — ningún import pisa silenciosamente un valor ya configurado en destino: genera una lista de conflictos explícita, resuelta por el usuario campo por campo. Para una herramienta pensada para que distintas instalaciones de distintas plantas intercambien catálogos de equipos entre sí, esa disciplina de versionado y de no-sobrescritura silenciosa es la diferencia entre un formato de intercambio confiable y uno que eventualmente corrompe datos de alguien en producción sin que nadie se entere hasta mucho después.

14. Comparativa: qué cambió realmente de la versión anterior

Para cerrar el análisis técnico, vale la pena poner en una sola tabla el contraste entre la arquitec-

tura VB.NET/WinForms original y la reescritura Python/GTK3 actual, tal como se puede reconstruir a partir de la documentación del proyecto y de las referencias explícitas del código a la versión anterior.

VB.NET/WinForms vs. Python/GTK3

- **Renderizado de diagramas:** generación de texto .dot → invocación de dot.exe externo → a veces re-parseo de la salida. *Ahora:* dibujo directo con Cairo sobre Gtk.DrawingArea, sin texto intermedio ni proceso externo.
- **Análisis de impacto:** sin evidencia de un motor de simulación de grafo equivalente en la versión anterior. *Ahora:* GraphImpactAnalyzer sobre GraphQLite (Cypher, Rust/C), con tres modos de simulación (cable, equipo, escenario combinado).
- **Dirección de señal (IN/OUT):** inferida de texto libre en el nombre del tipo de conector. *Ahora:* columna de control dedicada (tipo_conector.direccion), migrada con medición real de cobertura (14,4% de casos atípicos).
- **Persistencia:** SQLite en ambas versiones — la única pieza que cruzó intacta fue la vista CONEXIONES_AMBOS_EXTREMOS.
- **Internacionalización:** sin evidencia de soporte multi-idioma en la versión anterior. *Ahora:* español/inglés/portugués vía pipeline AST-based semi-automatizado.

No tuve acceso al código fuente de la versión VB.NET en sí — sólo a las referencias que el propio proyecto Python hace a ella en su documentación y comentarios — así que esta comparación está necesariamente limitada a lo que el equipo actual decidió documentar sobre su predecesora. Pero incluso con esa limitación, el patrón es claro: no fue una migración de sintaxis (reescribir el mismo diseño en otro lenguaje), fue una migración de *paradigma*, de una herramienta de documentación y dibujo a una herramienta de simulación y análisis.

15. Bajo el capó: la disciplina en modelo.py

Vale la pena detenerse en la capa de acceso a datos, porque ahí aparece uno de esos bugs de infraestructura

que son invisibles hasta que no lo son — y que la propia bitácora de aprendizajes del proyecto documenta explícitamente como una lección aprendida.

Listing 1: modelo.py — el fix de fuga de conexiones

```

1 @staticmethod
2 @contextlib.contextmanager
3 def _conn_ctx():
4     """Como _conn() pero como context manager
5     que SIEMPRE cierra la conexion al salir
6     (commit en exito, rollback en excepcion).
7     `with Modelo._conn() as conn:` en Python NO
8     cierra la conexion (solo hace commit/rollback
9     )
10    -> causaba fuga de conexiones SQLite que se
11    acumulaban en cada operacion, degradando
12    el rendimiento."""
13    conn = Modelo._conn()
14    try:
15        yield conn
16        conn.commit()
17    except Exception:
18        conn.rollback()
19        raise
20    finally:
21        conn.close()

```

El detalle es sutil y vale explicarlo para quien no lo haya pisado antes: en Python, el protocolo de context manager de sqlite3.Connection define `__exit__` para hacer commit o rollback automático — pero *no* para cerrar la conexión. Es un comportamiento documentado, pero contraintuitivo para cualquiera que venga de otros lenguajes donde “salir de un bloque with sobre una conexión” implica cerrarla. El patrón `with Modelo._conn() as conn:`, usado ingenuamente y repetido en cientos de call-sites a lo largo del proyecto, dejaba una conexión SQLite abierta por cada operación, acumulándose silenciosamente hasta degradar el rendimiento general de la aplicación con el uso prolongado — el tipo de bug que no aparece en una sesión de prueba corta y sí aparece después de un turno completo de trabajo real en planta.

La corrección no fue parchar cada call-site individualmente sino introducir `_conn_ctx()` como el único punto de verdad para el ciclo de vida de una conexión — abrir, commit-o-rollback según haya o no excepción, y cerrar siempre en el `finally` — y usarlo de ahí en más en toda consulta o escritura nueva. Es una solución de una sola función, con un docstring que explica el “por qué” del bug tan claramente como el “qué” del fix, exactamente la clase de comentario que ahorra que el mismo error se repita en seis meses cuando alguien — el mismo desarrollador o no — vuelva a escribir código de acceso a datos sin recordar por qué existe `_conn_ctx()` en primer lugar.

Sobre esa base, dos funciones utilitarias — `_query()`

para lectura y `_exec()` para escritura — son el punto de entrada que usa el resto de los casi 5.000 líneas de `modelo.py`, y por extensión toda la aplicación, para hablar con SQLite. Es una capa delgada a propósito: sin ORM, sin abstracción de por medio entre el SQL y el resultado, lo cual — para un dominio donde las consultas son tan específicas del negocio como las que arma `CONEXIONES_AMBOS_EXTREMOS` — es una elección razonable. Un ORM genérico difícilmente hubiera expresado con la misma naturalidad ese self-join con CTE que resuelve ambos extremos de un cable en una sola fila.

16. Import diferido: la cicatriz de un ciclo de dependencias real

Otro patrón que aparece documentado, con nombre y apellido, es el de *deferred imports* para romper dependencias circulares. La cadena concreta es `cabledoc.py` → `pantallas_avanzadas.py` → `senal_diagrama_ui.py`: el módulo principal importa las pantallas avanzadas, que a su vez — vía el mecanismo de mixins ya descripto — necesitan referenciar funcionalidad de diálogos que en algún punto vuelve a necesitar algo del módulo principal. La solución, consistente en todo el proyecto donde aparece este patrón, no es reestructurar el grafo de imports — que en un proyecto de este tamaño sería una tarea de alto riesgo y bajo beneficio inmediato — sino mover el `import` al interior de la función handler que efectivamente lo necesita, en vez de al tope del archivo:

Listing 2: Patrón de import diferido, tal como se repite en el proyecto

```
1 def _abrir_reportes_senal(self, *_a):
2     # Import diferido: evita el ciclo
3     # cabledoc -> pantallas_avanzadas ->
4     # senal_diagrama_ui en tiempo de carga
5     # de modulo.
6     from cabledoc import abrir_reportes_senal
7     abrir_reportes_senal(self)
```

Es una técnica conocida y, en general, considerada un parche más que una solución — un ciclo de imports suele ser síntoma de una frontera de responsabilidad mal trazada entre módulos. Pero en el contexto de una aplicación GTK3 de un solo proceso, sin necesidad de que esos módulos se puedan cargar de forma independiente para otro consumidor, es un costo aceptado a cambio de no forzar una reestructuración de módulos completa cada vez que se agrega un mixin nuevo al diagrama principal. Lo que hace correcto su uso acá no es la técnica en sí sino

la consistencia: se aplica siempre en el mismo punto de la cadena, documentado con el mismo comentario, en vez de aparecer como una solución ad-hoc distinta cada vez que se topa con el mismo problema.

17. Seguimiento de problemas: la memoria operativa de la planta

Un subsistema más chico pero con mucho valor operativo directo es el de seguimiento de problemas: las tablas `categoria_problema` y `problema_equipo` permiten registrar incidentes concretos contra un equipo — una fuente de alimentación que falló, un conector que empezó a dar falsos contactos — categorizados, no como texto libre suelto en un campo de notas. Ese historial no es un archivo aparte: alimenta directamente el cálculo de Probabilidad del Índice de Riesgo de Falla en `risk_engine.py` (`s_historical` en la fórmula de riesgo), lo cual cierra un círculo de retroalimentación real entre lo que pasó en planta y lo que el sistema recomienda vigilar de cerca a futuro.

Es un ejemplo más de un patrón que se repite en todo el proyecto y que vale la pena nombrar explícitamente porque es, para mí, el rasgo más distintivo de CableDoc frente a software de documentación de cableado genérico: cada entidad de datos que se agrega no queda aislada, sino que se integra activamente a los motores de análisis ya existentes. Un historial de problemas que sólo sirviera para consultarlo manualmente sería útil pero limitado; un historial de problemas que además mueve la aguja del score de riesgo de un equipo es una pieza de un sistema, no una feature aislada.

18. Testing sin red de seguridad tradicional

Vale dedicar un párrafo aparte a cómo se prueba este proyecto, porque es atípico y porque el propio changelog es transparente sobre sus límites. No hay una suite de `pytest` tradicional corriendo en CI para la capa de UI — lo cual, en un proyecto de este tamaño con GTK3 como framework, no sorprende: probar interacción real de mouse y widgets GTK de forma automatizada sigue siendo, en general, más costoso que el valor que aporta para un equipo de un solo desarrollador. En su lugar, la validación de UI se hace con GTK real corriendo bajo Xvfb (un servidor X virtual sin salida gráfica), instanciando las ventanas reales, disparando señales reales con

`GLib.idle_add()`, y verificando que no haya excepciones ni comportamiento inesperado — contra una copia de la base de datos real, nunca la original.

Es una estrategia de prueba honesta sobre sus propios límites: el changelog documenta explícitamente los casos donde *no* se pudo probar interacción real de mouse (sólo construcción de widgets) por falta de un compositor real disponible en el entorno de prueba, y al menos una sesión donde no se pudo instalar `graphqlite` en el sandbox y hubo que compensar ejecutando a mano, contra la base real, cada consulta SQL nueva que un módulo iba a usar — validando el contrato de datos aunque no se pudiera correr el motor de grafo completo end-to-end.

19. La reorganización de la barra de menús: cuando la UI empieza a pesar

Un capítulo del changelog que vale la pena mirar de cerca porque es un ejemplo perfecto de refactor de UI motivado por acumulación orgánica de funcionalidad: la ventana “Diagrama de conexiones” llegó a tener dos filas de aproximadamente cuarenta widgets sueltos — botones y toggles agregados uno por uno, feature tras feature, a medida que cada mixin nuevo (`ImpactoMixin`, `RiesgoDiagramaMixin`, `SenalDiagramaMixin`, `EscenarioMixin`) sumaba su propia fila de controles. Es exactamente el resultado esperable de un patrón de composición por mixins sin un punto de reorganización periódico: cada pieza individual tiene sentido, pero la suma se vuelve inmanejable para el usuario.

La solución fue reemplazar esas dos filas por una `Gtk.MenuBar` con ocho menús desplegables organizados por función — Ver, Buscar, Exportar, Impacto, Riesgo, Señal, Escenario, Herramientas — dejando debajo una fila delgada puramente informativa (selección actual, nivel de zoom) que ya no son controles interactivos. El cambio no fue sólo visual: implicó renombrar el método de integración de cada mixin de `_xxx_crear_botones()` a `_xxx_crear_items_menu()`, devolviendo `Gtk.MenuItem`/`Gtk.CheckMenuItem` en vez de `Gtk.Button`/`Gtk.ToggleButton` — un cambio de contrato que atraviesa cinco archivos distintos, coordinado en una sola entrega, con los handlers existentes reutilizados sin modificación porque `Gtk.CheckMenuItem` expone el mismo método `get_active()` que ya usaban

los toggles viejos. Ese detalle — elegir deliberadamente el nuevo tipo de widget porque expone la misma API que el viejo — es la clase de decisión que reduce drásticamente la superficie de riesgo de un refactor que, de otro modo, hubiera tocado cada handler de cada mixin.

Un efecto colateral documentado, del tipo que sólo aparece al migrar de botones fijos a menús desplegables: el popover de la leyenda de colores de señal se anclaba al widget que disparaba la acción, y ese widget — ahora un ítem de menú — se desmapea apenas el menú se cierra, así que el popover perdía su ancla y dejaba de funcionar. La corrección fue anclar el popover al canvas del diagrama en su lugar, que es un widget siempre presente. Y en una entrega posterior, esa misma leyenda de colores se rediseñó por completo, de un popover transitorio a un panel fijo dibujado directamente con Cairo sobre el canvas — mismo estilo visual que el panel de análisis de impacto, ubicado a propósito en la esquina superior izquierda para no superponerse con los paneles de impacto/escenario (que usan la esquina superior derecha) ni con el minimapa (esquina inferior derecha). Es un nivel de cuidado por la composición visual de una pantalla con múltiples overlays simultáneos que normalmente sólo se ve en herramientas con presupuesto de diseño dedicado, no en una aplicación interna de planta.

20. Detección de formato por contenido, no por extensión

Un detalle pequeño pero revelador de cuidado defensivo: el manejo de imágenes de referencia visual de señal en `senal_visual.py` no confía en la extensión del archivo para decidir si es un PNG válido. En cambio, detecta el formato real inspeccionando la firma de bytes del archivo (el *magic number* que identifica a un PNG independientemente de cómo se llame el archivo), y si el contenido real no es PNG, lo convierte sobre la marcha vía `GdkPixbuf` a un archivo temporal antes de usarlo donde el resto del sistema espera específicamente ese formato.

Es una decisión pequeña que evita una clase entera de bugs silenciosos: cualquier pipeline de gestión de activos de una planta real termina, tarde o temprano, recibiendo un archivo con extensión `.png` que en realidad es un JPEG renombrado a mano por alguien apurado, o un archivo corrupto con la extensión correcta pero el contenido roto. Validar por contenido en vez de por nombre — acompañado de una migración idempotente (`migrar_imagenes_senal_no_png()`) que corrige de

una vez los casos ya cargados en la base — es el tipo de robustez que no se nota hasta el día que evita un incidente, y que rara vez se prioriza a menos que alguien con experiencia real de campo esté diseñando el sistema.

21. Cinco preguntas que le haría al changelog

A modo de ejercicio de cierre, y porque una revista técnica que se precie siempre deja algo para pensar más allá del propio artículo, dejo cinco preguntas abiertas que — si tuviera acceso directo al desarrollador — me gustaría hacerle, no porque el proyecto esté incompleto sino porque son las preguntas naturales que deja cualquier sistema vivo de este tamaño:

1. ¿Hay planes de extraer un contrato de tipos explícito (dataclasses o TypedDict) para las filas que hoy circulan como `list[dict]` entre `_leer_bd()` y el resto de `graph_impact.py`? El propio motor ya usa dataclasses para sus resultados (`ResultadoImpacto`, `ResultadoEscenario`) — extender ese mismo criterio a las estructuras intermedias cerraría el círculo.
2. ¿CONEXIONES_AMBOS_EXTREMOS tiene, en algún horizonte, una versión con columnas nombradas de forma explícita (o una capa Python que la envuelva) para no depender de la posición ordinal en cada call-site?
3. ¿Cómo se decide, hoy, cuándo un fallback de compatibilidad hacia el criterio viejo (nombre de tipo de conector, prefijo de nombre) puede eliminarse con seguridad? ¿Hay un criterio de “*X días sin uso del camino de fallback*” medible, o es una decisión manual caso por caso como en la Fase C de función de patchera?
4. El asistente de diagnóstico por bisección es, en esencia, un algoritmo de búsqueda sobre un grafo con incertidumbre humana en cada paso (“no sé”). ¿Se evaluó ponderar la elección del próximo punto de test por accesibilidad física real (¿está en un rack a la altura de la vista, o hay que subir una escalera?) en vez de por posición estrictamente central en la cadena?
5. Con 44 tablas y un patrón de migración idempotente ya consolidado, ¿existe o se planea una herramienta de introspección que liste, para un desarrollador nuevo en el proyecto, qué migraciones ya corrieron y en qué orden — más allá de leer el changelog completo de punta a punta, que a esta altura roza las mil líneas?

Ninguna de estas preguntas es una crítica encubierta; son, honestamente, el tipo de pregunta que uno le hace a un colega cuyo trabajo respeta, para entender el próximo paso, no para cuestionar el paso ya dado.

22. Glosario mínimo para quien no viene del broadcast

Como esta nota puede terminar en manos de un lector con background más de software que de planta de transmisión, va un glosario breve de los términos de dominio que aparecen una y otra vez a lo largo del artículo, tal como se usan en el propio código y changelog del proyecto:

Vocabulario de planta

- **Patchera (*patch panel*):** panel de conectores donde cada cable de un equipo termina en un jack de atrás (*back*), permitiendo — opcionalmente — “interceptar” la señal por un jack de adelante (*front*) sin desconectar el cable original. Sin nada patcheado adelante, la señal pasa de largo (*bypass*); con un patchcord puesto adelante, se deriva o se inserta otra señal en su lugar.
- **DA (Distribution Amplifier):** equipo que toma una entrada y la replica en varias salidas idénticas, sin alterar la señal — el DISTRIBUIDOR del modelo de datos.
- **Matriz de ruteo:** equipo (o banco de equipos) que permite conectar cualquiera de N entradas a cualquiera de M salidas, reconfigurable sin recablear físicamente — el ENRUTADOR/MATRIZ del modelo.
- **DSK (Downstream Keyer):** compuerta de mezcla que combina una señal de fondo con un elemento gráfico superpuesto (un rótulo, un logo) — el caso de uso típico detrás del motor de reglas lógicas AND/OR del proyecto.
- **Cadena de aire:** el camino de señal que efectivamente sale al aire (transmisión), por oposición a rutas de monitoreo, respaldo o preview — lo que en CableDoc se marca como conjunto de “equipos críticos” para que el índice de riesgo priorice correctamente.
- **FANTASMA:** en el modelo de datos de CableDoc, un tipo de equipo que existe para completar

la documentación física (por ejemplo, una regleta de bornes o un punto de paso sin electrónica activa) pero que no debe considerarse en el análisis de impacto de señal — de ahí que el motor de grafo lo excluya explícitamente de los resultados.

23. Escala del proyecto en contexto

Para cerrar con perspectiva de magnitud: 44 tablas y 14 vistas SQL, ~32.000 líneas de Python repartidas en 19 módulos, un motor de grafos embebido con su propio lenguaje de consulta, un pipeline de traducción semi-automatizado con 334 claves, y — según los datos de producción que aparecen citados en el propio changelog — una base real con del orden de 414 equipos y 216 patcheras documentadas. No es un proyecto de juguete ni una prueba de concepto: es una aplicación en uso activo, sobre datos reales de una planta real, sostenida por un único desarrollador con un ritmo de entregas sorprendentemente alto — múltiples entregas documentadas en un mismo día, cada una con su propia validación contra datos reales y su propia entrada de changelog.

Esa combinación — alcance funcional amplio, disciplina de proceso sostenida, y velocidad de entrega alta — es la que hace que valga la pena escribir 20 páginas sobre un proyecto que, a primera vista, “sólo” documenta cableado. Documentar cableado bien, hasta el punto de poder simular su falla antes de que ocurra, es un problema de ingeniería genuinamente difícil, y CableDoc lo resuelve con un nivel de cuidado que rara vez se encuentra en herramientas de este nicho tan específico.

24. Reportes de señal: cuando el modelo empieza a auditarse solo

Vale dedicarle espacio a un conjunto de tres consultas que, aunque chicas en líneas de código, ejemplifican bien el momento en que un sistema de documentación deja de ser pasivo y empieza a señalar sus propias inconsistencias. `Modelo.reporte_formatos_en_uso()` cuenta, por cada formato de señal del catálogo, cuántos conectores y cuántas señales distintas lo usan hoy — pensado explícitamente, según el propio changelog, para decidir con datos reales (no con la memoria de alguien) dónde conviene priorizar una migración de infraestructura SDI a IP. `Modelo.reporte_senales_sin_usar()` encuentra señales dadas de alta en el catálogo que nunca llegaron a asignarse a ningún conector

— el equivalente a datos maestros huérfanos. Y `Modelo.reporte_senales_propagadas_sin_origen()` es, de las tres, la más interesante desde el punto de vista de integridad de datos: detecta señales que tienen al menos una fila de propagación automática pero *ninguna* carga manual que la sostenga — el rastro que deja alguien que borró o cambió una carga manual después de correr la propagación, dejando filas “propagadas” colgando de una fuente que ya no existe.

Ese tercer reporte es, en esencia, una prueba de consistencia referencial que SQLite — sin foreign keys diseñadas específicamente para ese caso, porque el origen “manual” y el origen “propagado” conviven en la misma tabla `senal_en_conector` — no puede garantizar por sí solo a nivel de esquema. El proyecto lo compensa con una consulta de auditoría explícita, expuesta en la UI como una de las tres pestañas de un diálogo de reportes, en vez de dejar que esa inconsistencia se acumule invisible hasta que alguien confíe en un dato de propagación que en realidad no tiene fuente real detrás. Es una forma barata — una consulta SQL bien pensada — de comprar bastante confianza en la integridad del propio modelo.

25. Rendimiento: medir antes de optimizar, con números reales

Ya mencioné de paso el caso más citable de optimización de rendimiento del proyecto — el patrón N+1 de `tiene_regla_logica_efectiva()` llamado una vez por nodo durante el render del diagrama, reemplazado por una consulta batch `equipos_con_regla_logica_activa()` — pero vale la pena detenerse en la magnitud real del cambio, porque no es un caso hipotético de manual de buenas prácticas: es una medición concreta, contra datos reales, de ~2,2 segundos a ~8 milisegundos. Eso es un factor de mejora de más de 250 veces, y la razón por la que el número es tan grande no es sutil: con 414 equipos en la base, un patrón N+1 significa 414 round-trips a SQLite — cada uno con su propio costo fijo de parseo de query y adquisición de conexión — en cada repintado del diagrama, mientras que la versión batch resuelve el mismo resultado en una sola consulta con un JOIN o un IN sobre el conjunto completo de ids.

Lo que hace esta optimización un buen caso de estudio, más allá del número en sí, es cuándo y cómo se detectó: no fue una sesión de profiling dedicada sino, con toda probabilidad, la reacción a una lentitud perceptible al usar

la propia aplicación — exactamente el tipo de señal que un desarrollador con experiencia de uso real de su propia herramienta capta antes que cualquier herramienta de profiling automatizado. Es un recordatorio simple pero fácil de olvidar bajo presión de entrega: en una aplicación de escritorio con un ciclo de render interactivo, un patrón de acceso a datos por nodo dentro de un loop de dibujo es, casi por definición, un candidato a revisar antes de que el problema se note en producción.

26. Un turno de guardia, reconstruido

Para que el resto del análisis no quede sólo en abstracciones de arquitectura, vale la pena reconstruir — a partir de lo que el propio código permite hacer, encadenando funcionalidades reales del proyecto — cómo se vería usar CableDoc en el momento exacto para el que fue pensado: no en un escritorio tranquilo planificando una migración, sino en medio de una guardia, con algo fuera de aire.

Son las tres de la mañana y control llama: se perdió la señal de un canal satelital. El técnico de guardia abre el Diagrama de conexiones, activa el modo “Diagnosticar falla” desde el submenú Escenario, y hace clic sobre el conector de salida del monitor donde se detectó el corte. El asistente reconstruye la cadena completa hacia la fuente — atravesando, sin que el técnico tenga que saberlo, dos patcheras en bypass total y una matriz con ruteo configurado — y arranca la bisección: pregunta por un punto de test a mitad de camino, físicamente accesible en un jack frontal de patchera. El técnico verifica con un tester portátil, contesta “No”, y la mitad correcta de la cadena queda descartada en un segundo paso. Tres preguntas después, el sistema aísla el tramo exacto: un cable entre un distribuidor y la primera patchera. No es magia — es una búsqueda binaria bien instrumentada sobre un grafo bien modelado — pero para alguien que de otro modo estaría siguiendo ese mismo cable físicamente por el rack con una linterna, la diferencia entre tres preguntas y cuarenta minutos de seguimiento manual es la diferencia entre resolver el problema antes o después de que termine el bloque de aire afectado.

Con el tramo identificado, el técnico decide que la reparación definitiva — resoldar el conector, o cambiar el cable — espera hasta la mañana, pero necesita una salida transitoria ya. Activa el “Modo escenario”, marca el cable sospechoso como cortado, y arrastra una conexión virtual de emergencia desde un puerto de reserva de

la misma patchera hasta el distribuidor. El panel lateral muestra de inmediato el comparativo antes/después: de siete equipos impactados por el corte, la reconexión virtual recupera los siete. Antes de tocar un solo cable físico, pide “Aplicar...”, que muestra el resumen exacto de qué se va a desconectar y qué se va a crear — sin sorpresas — y sólo entonces confirma. La reconexión queda registrada como cambio real en la base, con su propio código de cable temporal, lista para que el equipo de mantenimiento la revierta prolijamente cuando llegue el cable de reemplazo real.

Al cerrar la sesión de diagnóstico, el sistema ofrece guardar el resultado en el historial. El técnico lo hace, con una nota breve. Un mes después, cuando el mismo cable vuelve a dar problemas intermitentes, otro técnico — o el mismo, sin memoria fresca del incidente anterior — abre el “prontuario” desde el listado de cables, filtra por ese cable puntual, y encuentra dos sesiones de diagnóstico previas apuntando exactamente al mismo tramo. Ya no es un síntoma nuevo: es un patrón. Y ese patrón, alimentado de vuelta al historial de problemas del equipo, es exactamente el dato que el Índice de Riesgo de Falla necesita para dejar de tratar ese cable como uno más entre cuatrocientos y empezar a marcarlo como candidato prioritario de reemplazo preventivo, antes de que falle en un momento peor.

Ese recorrido — diagnóstico por bisección, simulación de contingencia antes de aplicar, historial consultable, y retroalimentación al score de riesgo — no es una lista de features independientes conectadas por el menú. Es un único flujo de trabajo que atraviesa cinco módulos distintos (`diagnostico_ui.py`, `escenario_ui.py`, `escenario_engine.py`, `graph_impact.py`, `risk_engine.py`) apoyados todos sobre el mismo modelo de grafo subyacente. Es, para mí, la mejor evidencia posible de que la inversión de tiempo en modelar bien la topología — direcciones explícitas, patcheras como sub-grafo, reglas lógicas con prioridad correcta — no fue un ejercicio académico de arquitectura limpia por la arquitectura misma, sino la base que hizo posible construir, sobre ella, herramientas operativas que un técnico real puede usar bajo presión real.

27. Ritmo de entrega y versionado

Un último dato de proceso que vale la pena registrar antes del cierre: la aplicación versiona con un esquema de timestamp (`APP_VERSION`, formato `a.aaamddhhmmss`) actualizado en cada entrega, visible desde el diálogo “Ac-

erca de...” junto al historial completo de changelog renderizado en Markdown básico dentro de la propia aplicación — sin depender de que el usuario final sepa dónde está el archivo `changelog.txt` en el disco, o siquiera que existe. Es una decisión chica pero con una implicancia clara: cualquier usuario de CableDoc, no sólo el desarrollador, puede en cualquier momento abrir la aplicación y ver exactamente qué versión tiene corriendo y qué cambió desde la anterior, con el mismo nivel de detalle que un desarrollador leyendo el changelog crudo. Para una herramienta que corre en producción real, sin un canal de release notes separado ni un sitio de documentación externo, ese diálogo “Acerca de...” termina cumpliendo la función que en un proyecto más grande cumpliría un changelog público versionado en un repositorio — resuelto acá con el mínimo de infraestructura posible: un archivo de texto y un parser de Markdown de cien líneas.

28. Autocrítica con herramienta: el informe DRY

El hallazgo más revelador sobre la salud interna del código no lo encontré yo leyendo el proyecto — lo encontró el propio proyecto sobre sí mismo. Entre los documentos de planificación hay un `informe_dry_cabledoc.md`, resultado de correr `jscpd` (una herramienta de detección de clones de código consciente de AST, no un simple diff de texto) sobre 15 archivos y unas 29.253 líneas. El resultado: 114 clones detectados, 1.732 líneas duplicadas (5,9% del total analizado), y un patrón dominante que el propio informe describe con una frase que vale citar por su precisión: *“no es código repetido entre módulos, es el mismo archivo repitiéndose consigo mismo”* — 112 de los 114 clones son duplicación *intra-archivo*, no entre archivos distintos.

La causa raíz que identifica el informe es consistente con algo que ya veníamos viendo en la separación catálogo/instancia: cada vez que se agregó el “molde” de catálogo para una entidad que ya existía como instancia real — `equipo/equipo_catalogo`, `slot/slot_catalogo`, `conector/conector_catalogo` — la clase de UI correspondiente se copió y renombró en vez de generalizarse con un parámetro de tabla. El ejemplo más grande documentado son `_DialogoAltaRapidaEquipo` y `_DialogoAltaRapidaCatalogo` en `cabledoc.py`: 165 líneas de `__init__` prácticamente idénticas, más varios métodos completos duplicados letra por letra,

sumando cerca de 300 líneas de la nota de mayor impacto del informe.

Lo interesante — y lo que hace que valga la pena incluir esta sección en vez de omitirla por ser una crítica — es cómo se procesó el hallazgo. El informe no se queda en el diagnóstico: propone un plan de acción concreto, ordenado por relación impacto/riesgo, con siete ítems independientes (extraer una clase base parametrizada por diccionario de nombres de columna, un helper único de exportación PDF/SVG, un constructor genérico de panel de pendientes, y así) cada uno con su estimación de líneas a eliminar y su nivel de riesgo explícito. La estimación total, si se aplicara el plan completo, es de ~1.170 líneas eliminadas — cerca de dos tercios de la duplicación detectada — dejando deliberadamente afuera los fragmentos menores de 10–15 líneas, *“no siempre valen la pena refactorizar”*, en palabras del propio informe.

Por qué esto importa más que el número: un proyecto que corre una herramienta de detección de clones sobre sí mismo, documenta el resultado con la misma seriedad que un bug de producción, y prioriza el refactor por riesgo en vez de por tamaño, está tratando la deuda técnica como un ítem de backlog gestionable -- no como algo que se descubre recién cuando ya duele. A la fecha de este review, el plan figura como propuesta: es deuda identificada y priorizada, no todavía pagada, lo cual es una distinción honesta que vale la pena remarcar en vez de dar por hecho que un informe implica que ya se resolvió.

29. Lo que viene: el mapa de plans/

La carpeta `plans/` del proyecto — diecinueve documentos de planificación, algunos ya ejecutados y absorbidos en el changelog, otros todavía en estado de propuesta — funciona como una suerte de RFC interno: cada feature de peso pasa primero por un documento de diseño explícito antes de tocar código. Uno de los que sigue sin implementar al momento de este review, `plan_impacto_perdida_energia_rack.md`, es un buen cierre para pensar hacia dónde puede crecer el motor de análisis: la idea es generalizar el análisis de impacto — hoy pensado para un cable o un equipo a la vez — a la pregunta *“si se corta la energía de este rack completo, ¿qué cadena de señal se cae?”*, cortando de una sola vez

todos los cables que tocan cualquier equipo físicamente montado en ese rack.

Lo notable del plan, incluso sin estar implementado, es que ya deja explícito por qué es barato de construir sobre la base actual: no requiere ninguna tabla nueva ni cambio de esquema, porque toda la información necesaria — qué equipo está en qué rack, qué conectores tiene, cómo se resuelven sus dos extremos — ya existe en `posicion_en_rack`, `conector` y la vieja conocida `CONEXIONES_AMBOS_EXTREMOS`. Es la mejor demostración posible de que invertir en un modelo de datos bien pensado desde el principio no es un costo que se paga una vez: es un activo que sigue devengando funcionalidad nueva, entrega tras entrega, mucho después de que el esquema original se escribió.

30. El semáforo de riesgo, en detalle

Antes de cerrar, vale la pena mostrar cómo se traduce el Índice de Riesgo de Falla en algo que un técnico puede leer de un vistazo en el diagrama. El cálculo produce un número entre 0 y 100, pero lo que efectivamente se pinta en pantalla — coloreando cada equipo del diagrama según su nivel — es una clasificación en cuatro bandas con su propio color asociado, definidas como una lista ordenada de umbrales en `risk_engine.py`:

Niveles de riesgo (IRF)

- **Crítico** (≥ 75): rojo intenso — intervención prioritaria
- **Alto** (≥ 50): naranja — seguimiento cercano
- **Medio** (≥ 25): amarillo — vigilar tendencia
- **Bajo** (< 25): verde — sin acción requerida

La implementación de esa clasificación es deliberadamente aburrida — en el buen sentido — y por eso vale mostrarla: una función `nivel_de(riesgo)` que recorre la lista de umbrales de mayor a menor y devuelve el primero que el valor supera, con una función hermana `color_de(riesgo)` que hace exactamente lo mismo pero devuelve la terna RGB en vez del nombre. No hay una tabla de CASE WHEN duplicada en SQL, ni una constante repetida entre el motor de cálculo y la capa de dibujo: ambas funciones leen de la misma lista NIVELES, así que cambiar un umbral — decidir que “Crítico” debería empezar en 70 en vez de 75, por ejemplo — es un cambio de una sola línea que se propaga automáticamente

tanto al texto mostrado como al color pintado en el diagrama. Es un ejemplo pequeño pero representativo de un principio que aparece una y otra vez en el proyecto: cuando dos partes del sistema necesitan estar de acuerdo sobre un mismo criterio, que lean literalmente la misma fuente de verdad, en vez de mantener el criterio duplicado en dos lugares y confiar en que alguien se acuerde de actualizar ambos a la vez.

Y el propio docstring del motor de riesgo es honesto sobre su propia degradación elegante: si GraphQLite no está disponible en el entorno — o el equipo evaluado no tiene ningún cable cargado todavía — el factor Impacto cae a un valor neutro configurable en vez de hacer fallar el cálculo completo. Es la misma filosofía defensiva que ya vimos en la detección de PNG por contenido de bytes o en el manejo de tablas de señal todavía no creadas: preferir un resultado parcial pero disponible antes que una excepción que deja al usuario sin nada.

31. Dos propuestas que muestran madurez de dominio

Cierro el recorrido por `plans/` con dos documentos todavía sin implementar que, aunque nunca lleguen a escribirse en código, ya demuestran algo valioso: la capacidad de traducir un problema real de planta en un modelo matemático concreto antes de tocar una sola línea.

El segundo, `plan_riesgo_senal_audio.md`, era — al momento del relevamiento original de este review — el más ambicioso de los dos planes sin implementar: proponía extender el análisis de impacto *lógico* (¿qué se desconecta?) con un análisis de riesgo de *calidad* de señal — cadenas que están conectadas y funcionan, pero son eléctricamente frágiles. Como se detalla en la sección de actualización más adelante, ese plan ya se implementó por completo entre la redacción de este review y su publicación, así que lo dejo documentado acá como estaba planteado — para mostrar la calidad del razonamiento previo a escribir código — y remito a la sección §“Actualización: la entrega del 20 de agosto” para el resultado real.

El documento descompone el problema en tres riesgos con matemática distinta pero calculables sobre el mismo grafo que ya usa `GraphImpactAnalyzer`: atenuación acumulada en tramos analógicos largos (una suma a lo largo de la cadena), cuello de botella de ancho de banda (un mínimo entre eslabones, no una suma), y mismatch de formato o balance eléctrico entre tramos consecutivos

(una comparación puntual entre pares). Lo que hace bien pensado al planteo es reconocer explícitamente que las tres preguntas comparten la misma estructura de recorrido de grafo — equipos como nodos, cables como aristas dirigidas — y sólo cambian en la operación de agregación aplicada sobre ese recorrido. Es, otra vez, el mismo principio de reutilización estructural que ya vimos en el grafo invertido de `senal_visual.py`: no tres motores nuevos, sino tres formas distintas de recorrer el motor que ya existe.

El primero, `plan_sugerencia_fusion_cables_por_metroaje`, no se conforma con resolver el caso feliz. Se pregunta sigue sin implementar a la fecha de esta actualización, y sigue siendo un buen ejemplo de traducir un problema de reconocimiento físico en un problema de coincidencia numérica entre registros: si dos puntas sueltas de un relevamiento son en verdad el mismo cable físico, la diferencia entre sus dos lecturas de metraje impreso en la funda debería aproximar la longitud real del tramo — una heurística puramente aritmética, sin cámaras ni tester de continuidad.

32. Vista previa visual: la pregunta que nadie más se hace

Falta cubrir un módulo que, por su nombre, podría pasar por un capricho estético y que en realidad resuelve un problema real de operación en vivo: `senal_visual_ui.py` (620 líneas), la interfaz sobre el motor descrito más arriba en `senal_visual.py`. La pregunta que responde es distinta de todas las anteriores — no “¿qué se cae?”, no “¿qué nombre tiene la señal?”, sino “¿qué imagen de referencia le corresponde a este conector de salida *en este momento*?”. Es una pregunta que sólo tiene sentido en un contexto muy puntual: control de calidad visual en tiempo real, donde un operador necesita confirmar de un vistazo que lo que sale por un conector determinado es efectivamente la señal esperada, sin tener que memorizar una tabla de equivalencias ni seguir el cable físicamente hasta un monitor.

El mecanismo de resolución — ya descrito en la sección del grafo invertido — permite que esa imagen se resuelva de forma dinámica y en cascada: si el conector tiene una estrategia visual propia o heredada de su tipo de equipo, se compone; si no, si tiene una imagen cargada a mano, se usa esa; si no, se sigue recursivamente hacia quien lo alimenta; y si nada de eso resuelve, se informa explícitamente el motivo por el que no hay imagen disponible, en vez de fallar en silencio o mostrar un

placeholder genérico sin contexto. Esa última decisión — devolver siempre un motivo explícito cuando no hay resultado, en vez de un simple `None` — es pequeña pero significativa: convierte cada ausencia de imagen en información accionable (“no tiene imagen manual y nada lo alimenta” es un diagnóstico distinto de “tiene imagen pero el archivo no se pudo abrir”), en vez de dejar al usuario adivinando por qué la vista previa está vacía.

Es, otra vez, la misma señal que atraviesa todo este review: cada subsistema nuevo que se agrega a `CableDoc` no se conforma con resolver el caso feliz. Se pregunta qué pasa cuando la respuesta es “no sé”, y responde esa pregunta con la misma seriedad que la principal.

33. Entrega segura: cómo se protege el propio proceso de trabajo

Un último punto, más de proceso que de código, pero que en un proyecto mantenido con asistencia de herramientas de IA para gran parte de la implementación merece mención explícita: la disciplina de entrega documentada evita sistemáticamente el riesgo más común de ese modo de trabajo, que es el sobrescrito ciego. El propio flujo de trabajo documentado reconoce que el filesystem real de Pototo suele tener desarrollo en paralelo no reflejado en una copia exportada del proyecto — los archivos reales son, en general, medibles como más grandes que los de cualquier zip de referencia — así que una sobrescritura completa de archivo es tratada explícitamente como una operación riesgosa. La alternativa adoptada en la práctica es doble: entregas empaquetadas como archivos comprimidos que contienen únicamente los archivos efectivamente tocados, nunca el árbol completo, o — cuando la conexión directa al filesystem está disponible y estable — reemplazos de texto puntuales con verificación de coincidencia exacta antes de aplicar cualquier cambio, y una vista previa en seco cuando el cambio lo amerita.

Es una precaución que cuesta poco y que evita el peor escenario posible en un proyecto de un solo desarrollador sin control de versiones centralizado activamente monitoreado en cada sesión: perder trabajo propio no versionado por sobrescribirlo accidentalmente con una versión más vieja traída de otro lado. Vale la pena nombrarlo explícitamente en este review porque es, en el fondo, la misma disciplina que ya vimos aplicada una y otra vez al propio dominio del proyecto — nunca pisar un valor local sin preguntar, ya sea que ese valor viva en una fila de

tipo_conector o en un archivo Python en disco.

34. Fichas de referencia rápida

Para el lector que llegó hasta acá buscando algo consultable después, más que una lectura corrida, va un resumen de los módulos que sostienen el análisis de señal — la columna vertebral de todo lo descrito en este review — con su responsabilidad exacta en una línea.

Mapa de módulos del núcleo de análisis

- `graph_impact.py` — construye el grafo dirigido completo y responde impacto de cortes, fallas de equipo y escenarios combinados.
- `risk_engine.py` — Índice de Riesgo de Falla por equipo, combinando probabilidad e impacto medido sobre el grafo anterior.
- `senal_propagation.py` — motor de propagación de nombre de señal (Bellman-Ford, sin GraphQLite), responde *qué* señal circula por cada conector.
- `senal_visual.py` — grafo invertido para resolver *qué imagen* de referencia corresponde a un conector en este momento.
- `diagnostico_falla.py` — bisección real sobre la cadena entre síntoma y fuente, clasificando por qué se corta.
- `escenario_engine.py` — orquesta simulación y aplicación real de cambios combinados, sin GTK.
- `cypher_console.py` — acceso directo al lenguaje de consulta del motor de grafos, para el usuario avanzado.

Los siete módulos comparten una misma decisión de diseño que atraviesa todo este review y que vale la pena remarcar una última vez: ninguno reimplementa su propio recorrido de grafo desde cero. Todos leen la misma noción de “quién alimenta a quién”, construida una sola vez a partir de las columnas de control (`direccion`, `rol_senal`, `id_funcion_patcher`) que la Fase 1 a 9 del plan de eliminación de hardcodes de idioma dejó como la única fuente de verdad. Es la prueba más concreta de que ese trabajo de migración — tedioso, sin funcionalidad visible nueva para el usuario final el día que se entregó — fue la inversión que hizo posible construir,

sobre una base común, cada una de las herramientas operativas descritas en las secciones anteriores.

35. Cierre

Escribo esta nota con la misma vara que aplicaría a cualquier sistema que vaya a sostener una guardia real de aire: no si el código es elegante en abstracto, sino si el técnico que atiende el llamado de las tres de la mañana confía en lo que la pantalla le dice. Por lo que pude relevar del código, del changelog y de los propios documentos de planificación de CableDoc, esa confianza está bien depositada — no porque el sistema esté libre de bugs (los dos que documenté en detalle bastan para desmentir esa idea), sino porque cada bug encontrado se investigó hasta la causa raíz, se corrigió con una validación medible contra datos reales, y se dejó por escrito para que el próximo que toque ese código no tenga que redescubrir el mismo problema desde cero. Esa, más que cualquier elección puntual de framework o de motor de grafos, es la característica que hace que valga la pena seguir construyendo sobre esta base.

36. Sincronización sin servidor: Google Drive vía rclone

Una limitación estructural de la decisión de usar SQLite como única base de datos — excelente para simplicidad de despliegue, sin proceso de servidor que administrar — es que el archivo vive en un único disco local. Para una herramienta pensada para consultarse desde más de un puesto de trabajo, o desde el escritorio de planta y — como se detalla en la sección siguiente — desde un celular, esa limitación necesita algún tipo de solución de sincronización sin caer en la complejidad de montar un servidor de base de datos real sólo para este propósito.

La solución adoptada es deliberadamente de bajo costo de mantenimiento: `rclone`, la herramienta de línea de comandos que trata a proveedores de almacenamiento en la nube como si fueran un sistema de archivos remoto más, sincronizando la carpeta `database/` completa contra Google Drive:

```
rclone copy -P -vv /home/sa/Desktop/cabledoc/
database/ \
remote:CableDocSync/database
```

Es una elección de infraestructura coherente con el resto de las decisiones de arquitectura del proyecto: nada de infraestructura propia que mantener — ni un servidor

de sincronización, ni una API de replicación — apoyándose en cambio en una herramienta madura y ya probada para el problema específico de mover archivos entre un disco local y un backend de nube. El costo de esa simplicidad es el que cualquiera que haya sincronizado un archivo SQLite entre dispositivos conoce bien: no hay resolución de conflictos a nivel de fila como la que ofrecería una base de datos con replicación real, así que el flujo de trabajo implícito es de un solo escritor activo por vez — sincronizar, trabajar, volver a sincronizar — más que edición concurrente desde dos puntos a la vez. Para el caso de uso real (un técnico que documenta desde el escritorio de planta y otro que consulta o hace ajustes puntuales desde el celular en el rack, no dos personas editando el mismo registro al mismo tiempo) es una limitación aceptable a cambio de no tener que operar ni un gramo de infraestructura de servidor adicional.

Lo notable es que ese mismo mecanismo — `rclone` apuntando al mismo recurso remoto de Google Drive — es también el puente hacia el lado móvil de este sistema. En el celular, la sincronización corre a través de **RoundSync**, una aplicación Android que empaqueta `rclone` con una interfaz gráfica y programación de tareas, en vez de requerir una terminal Linux o Termux para invocar el binario directamente. Configurada contra el mismo `remote:CableDocSync/database` que usa el escritorio, RoundSync le da al dispositivo móvil la misma vía de sincronización que ya usa la planta, sin duplicar la lógica de transporte ni depender de un servicio de sincronización distinto para cada plataforma. El resultado práctico es un único circuito — escritorio ↔ Google Drive ↔ celular — con `rclone` como denominador común en ambos extremos, y el mismo régimen de un-solo-escritor-activo aplicando igual de los dos lados: sincronizar antes de empezar a trabajar en el dispositivo que se vaya a usar, y volver a sincronizar al terminar, para no pisar en el otro extremo cambios que todavía no bajaron.

37. La versión de bolsillo: Kivy sobre Pydroid 3

El desarrollo más reciente y, para esta nota, el más inesperado: existe una migración completa de CableDoc a **Kivy**, el framework de UI multiplataforma de Python, pensada específicamente para correr en un teléfono Android a través de **Pydroid 3** — el intérprete de Python para Android que permite ejecutar y depurar scripts di-

rectamente en el dispositivo, sin pasar por un proceso de compilación de APK tradicional. No es un prototipo ni una prueba de concepto abandonada a mitad de camino: al momento de este agregado, la migración documenta seis fases completas y probadas contra datos reales, con una séptima fase explícitamente marcada como opcional y de prioridad baja.

CableDoc Kivy --- estado relevado

- **Alcance:** 100% de los módulos CRUD y las 3 vistas gráficas principales (rack, frame/slots, patcheras) migrados y probados
- **Código:** ~15.200 líneas en 20 archivos Python
- **Reutilización directa, sin cambios:** `modelo.py` (capa de datos completa) e `i18n.py`
- **Pendiente (fase 7, prioridad baja):** `DiagramaConexiones` (el diagrama de impacto completo) y los editores masivos de coordenadas

Lo primero que llama la atención, y que dice mucho de la calidad del desacople original entre capa de datos y capa de presentación en la versión GTK3, es que `modelo.py` — las casi 5.000 líneas de acceso a SQLite que sostienen toda la aplicación de escritorio — se reutiliza en la versión Kivy **sin ninguna modificación**. La misma base de datos `db.db`, el mismo esquema de 44 tablas, las mismas consultas parametrizadas: lo único que cambió de punta a punta fue la capa de presentación. Es la prueba práctica, más allá de cualquier argumento teórico sobre separación de responsabilidades, de que valió la pena mantener esa frontera limpia durante años de desarrollo de la versión de escritorio: cuando llegó el momento de portar a un framework de UI completamente distinto, sobre un sistema operativo completamente distinto, la mitad más grande y más crítica del código — la que garantiza la integridad del modelo de datos — no tuvo que tocarse.

El dato que más vale destacar: `modelo.py` sobrevivió intacto a *dos* reescrituras completas de interfaz --- GTK3 primero, Kivy ahora --- exactamente por la misma razón que `CONEXIONES_AMBOS_EXTREMOS` sobrevivió a la reescritura desde VB.NET. Cuando el modelo de datos y sus consultas están bien resueltos, se convierten en el activo más estable del

proyecto, no en el que hay que reescribir cada vez que cambia el framework de turno.

37.1 Lo que sí cambió: de bloqueante a callbacks

El cambio de arquitectura más profundo entre las dos interfaces no es cosmético, es de modelo de concurrencia de UI. `Gtk.Dialog.run()` en GTK3 es *bloqueante*: la ejecución se detiene ahí mismo hasta que el usuario responde, lo que permite código lineal y sincrónico (`if dlg.run() == Gtk.ResponseType.OK: guardar(...)`). Kivy no tiene equivalente bloqueante: todo `Popup` se abre con `.open()`, que retorna de inmediato, y la respuesta del usuario llega más tarde por un `callback`. La migración documenta haber aplicado esa transformación — de flujo lineal a flujo por `callback` — de forma consistente en absolutamente todos los diálogos del proyecto, un cambio de paradigma de control de flujo que atraviesa cada uno de los veinte archivos, no un parche puntual en un par de pantallas.

La tabla de equivalencias que documenta el propio README de migración es, en sí misma, un buen resumen de cuánto trabajo de traducción de conceptos — no sólo de sintaxis — implica portar una aplicación GTK3 madura a Kivy: `Gtk.TreeView` se convierte en una tabla armada a mano con `BoxLayout` dentro de un `ScrollView`; el renderizado Cairo directo sobre `Gtk.DrawingArea` se reemplaza por primitivas de `kivy.graphics` (`Ellipse`, `Rectangle`, `Line`) sin ningún motor de dibujo vectorial de por medio; y hasta el efecto de *glow* alrededor de un punto de conexión activo en la vista de patcheras — resuelto en Cairo con `cr.arc()` y un canal alfa — se reconstruye en Kivy con dos elipses semitransparentes superpuestas. Ningún atajo: cada primitiva visual del original tiene su contraparte deliberadamente pensada en el nuevo framework, no una aproximación genérica.

37.2 Detalles que sólo aparecen cuando el dispositivo es real

Lo que distingue a esta migración de un ejercicio de traducción de framework en el vacío es la cantidad de ajustes que sólo tienen sentido si el código efectivamente se probó en un teléfono real, no sólo en el emulador de escritorio. `main.py` detecta la plataforma en tiempo de ejecución (`kivy.utils.platform`) y, sólo en Android, fuerza orientación retrato y activa un modo inmersivo

que oculta la barra de estado y los botones de navegación del sistema — usando `pyjnius` para llamar directamente a la API nativa de Android (`PythonActivity`, `View.OnSystemUiVisibilityChangeListener`) cuando ese modo inmersivo se pierde después de un cambio de foco. El propio código documenta, con el mismo nivel de honestidad que vimos en el resto del proyecto, una falla de threading real encontrada durante las pruebas: el listener de Android corre en un hilo nativo distinto al hilo donde vive el código Python de Kivy, y una llamada directa se perdía en silencio sin ningún `traceback` visible — corregida despachando la llamada al hilo de UI correcto vía `android.runnable.run_on_ui_thread`.

También aparece un detalle pequeño pero revelador de cuidado por el entorno real de destino: los íconos de la interfaz de escritorio usan glifos Unicode simples — flechas, símbolos de navegación — que *no se renderizan* en la consola de Pydroid 3. La solución fue generar un set completo de íconos PNG propios (`generar_íconos.py`, dibujados a 4x de resolución con PIL y reducidos con antialiasing, en blanco puro sobre fondo transparente) que Kivy tiñe en tiempo de ejecución según el color correspondiente al tema claro u oscuro activo — un solo archivo de ícono sirviendo para ambos temas, en vez de duplicar assets. Y para diagnosticar problemas que sólo se manifiestan en el dispositivo — donde no hay una terminal a mano para ver un `traceback` en vivo — `logger_cabledoc.py` instala un hook global de excepciones no capturadas que escribe cada error, con fecha, origen y `traceback` completo, a un `log.txt` en la misma carpeta que la app, con el mismo criterio defensivo de “el logueo nunca debe romper la app” (cada escritura de log está, a su vez, envuelta en su propio `try/except` silencioso).

Con las fases 1 a 6 completas, lo único pendiente y explícitamente marcado como de prioridad baja es el diagrama de impacto interactivo completo — que depende del motor de análisis de grafo y merecería, según el propio plan de continuación documentado, una versión simplificada con `kivy.graphics.Line` en vez de un port directo de la lógica de layout automático que GTK3 heredaba en parte de `Graphviz` — y los editores masivos de coordenadas, cuya edición individual ya funciona en la versión móvil aunque todavía falta la variante de selección múltiple. Para el caso de uso que motivó la migración — consultar y hacer ajustes puntuales de documentación desde el rack, sin cargar una laptop — ese 100% de cobertura CRUD y las tres vistas gráficas principales ya cubren

el flujo de trabajo real de un técnico en planta; lo que queda pendiente es, en la práctica, la herramienta de planificación más pesada, que de todos modos tiene más sentido usar desde un escritorio con pantalla grande.

38. Actualización: la entrega del 20 de agosto

Entre la redacción original de este review y su publicación, el proyecto sumó tres desarrollos de peso en apenas 48 horas de trabajo, documentados con el mismo nivel de detalle — causa raíz, validación contra datos reales, número de versión — que ya elogíé en las secciones anteriores. Vale la pena incorporarlos porque dos de ellos cierran, de forma directa, líneas que este mismo artículo había dejado abiertas.

38.1 El riesgo de señal de audio, ya no es un plan: es `signal_risk.py`

El plan que analicé en la sección anterior como ejercicio de buen razonamiento de dominio sin código detrás dejó de ser eso: está implementado completo, con sus tres ejes, su UI y su exposición en Cypher. `SignalRiskAnalyzer` calcula, para cada cable, atenuación (marcada por bandera, sin acumulación de cadena en esta primera versión), cuello de botella de ancho de banda (comparación local contra los cables vecinos que comparten equipo) y mismatch de formato eléctrico (conductores, balance, canal entre los dos extremos), devolviendo los tres ejes siempre por separado — nunca fusionados en un único score — para que la UI pueda mostrar la causa específica en vez de un número opaco.

El detalle de esquema más interesante de esta entrega es uno que el plan original, según el propio changelog, no había anticipado: conector no tenía ninguna columna que describiera su formato eléctrico real — `tipo_conector` es un catálogo de forma física, no de formato de señal — así que hubo que agregar `conector.id_tipo_ficha` como una foreign key nueva a una tabla de fichas eléctricas (`tipo_ficha`: número de conductores, modo de balance y de canal por defecto, ancho de banda). El motor fue validado, según el propio changelog, contra un escenario sintético diseñado a propósito para ejercitar cada eje — una consola a grabador con un cable analógico de 120 metros marcando atenuación y balance a la vez, un router de video con un patchcord viejo marcando ancho de banda, una conexión TS a TRS marcando mismatch eléctrico — y contra los

496 cables reales de la base sin ningún falso positivo.

Un bug de diseño post-entrega, corregido apenas un día después y documentado con la misma honestidad de siempre: el primer intento de modelar la ficha eléctrica de un cable la puso como una columna única en cable — un solo valor para todo el cable completo. Un caso real planteado por el propio usuario lo desmintió: un cable puede tener una ficha XLR3 macho de un lado y una ficha TRS del otro, algo perfectamente normal en un adaptador de campo, que un solo campo por cable no puede representar. La corrección — `conexion.id_tipo_ficha`, nullable, uno por cada fila de conexión — es la misma idea que ya usa el proyecto en otros lugares: cuando dos extremos de una misma entidad pueden diferir, el dato correcto no vive en la entidad compartida sino en cada punto de conexión, que ya sabe sin ambigüedad a qué conector corresponde. El changelog registra explícitamente haber considerado y descartado la alternativa obvia — `cable.id_tipo_ficha_extremo_a/b` — por una razón sutil y bien identificada: no existe ningún ordinal estable que distinga “extremo A” de “extremo B”, porque la propia vista `CONEXIONES_AMBOS_EXTREMOS` arma esa etiqueta con un self-join sin ORDER BY, así que cualquier cosa que dependiera de esa distinción sería no determinística por construcción.

38.2 Bitácora de incidentes y riesgo analógico

El segundo desarrollo es un subsistema nuevo de punta a punta, sin antecedente directo en ninguno de los planes que había relevado para la edición anterior: una bitácora de incidentes (`incidente`, `zona_sospechosa` como conjunto reutilizable de equipos, y su vínculo con equipos/cables/zonas) combinada con un flag de “armado correcto” a nivel de conector y de cable, agregados en `riesgo_analogico.py` en un score de *zona caliente* por equipo, cable o zona — con decaimiento lineal por antigüedad dentro de una ventana configurable, y con el mismo criterio que ya vimos en `risk_engine.py` de nunca perder el detalle textual de cada factor que compone el número final.

Lo que distingue a este motor de sus primos — `GraphImpactAnalyzer` y `SignalRiskAnalyzer` — es una decisión de alcance documentada explícitamente en su propio docstring: no construye ni recorre el grafo de señal. Es agregación pura por equipo/cable/zona, sin propagación, a propósito, con un TODO marcado para una entrega futura donde se sumará la salida de

`signal_risk.py` al mismo score. Es una secuenciación de trabajo sensata — resolver primero el caso simple y verificarlo contra datos reales, dejar la integración con el grafo para una segunda entrega bien delimitada — en vez de intentar construir las dos capas de análisis a la vez y arriesgar no terminar ninguna con solidez.

El caso de bug más rico de esta entrega, otra vez motivado por una foto real que trajo el usuario, repite casi al pie de la letra el patrón de granularidad que ya resolvió el mismatch eléctrico unas horas antes: el sistema de “armado correcto” original vivía a nivel de cable completo (un flag para todo el cable) y a nivel de conector del equipo (el jack, no el cable) — ninguno de los dos podía representar “esta punta específica de este cable puntual está mal armada”, que es exactamente el caso que trajo el cliente: un cable con ficha XLR3 de un lado y TRS del otro, con el técnico habiendo cruzado conductores en una sola de las dos fichas, invisible a simple vista. La solución, consistente con la del mismatch eléctrico apenas un día antes, fue extender el mismo patrón un nivel más abajo: `conexion.es_armado_correcto/detalle_armado`, por fila de conexión, dejando el flag a nivel de cable completo intacto como un atajo rápido independiente. El changelog documenta haber descartado primero una tabla nueva (`problema_ficha_cable`) al notar la superposición con el sistema de armado ya existente — evitar crear una tabla paralela quando el patrón correcto era extender una ya existente es, otra vez, la misma disciplina de no duplicar una noción de estado que ya vimos en el capítulo de deuda técnica DRY de este mismo proyecto.

38.3 Ajustes finos al asistente de diagnóstico

Un tercer paquete de cambios, más chico pero con impacto directo en la usabilidad del asistente de diagnóstico por bisección ya descrito en este review: el diálogo del wizard dejó de ser modal, así que ya no bloquea el paneo y el zoom del diagrama mientras está abierto — `Gtk.Dialog.run()` sigue bloqueando el flujo de *código* Python hasta que el usuario responde, pero ya no bloquea la *interacción* con el resto de la ventana, una distinción sutil entre bloqueo de control de flujo y bloqueo de UI que vale la pena remarcar porque es fácil de confundir. Además, cada pregunta nueva del asistente ahora centra automáticamente el diagrama en el equipo correspondiente, reutilizando la lógica de cámara que ya existía para otros propósitos en vez de escribir un cálculo de paneo

nuevo. Y se relajó — a pedido explícito del cliente, tras confirmar que “colorear por señal” nunca había estado excluido por ser un overlay pasivo — la exclusión mutua entre el modo de diagnóstico y la vista previa de imagen, con un flag de sesión activa nuevo para evitar que un clic accidental en otro puerto abra una segunda sesión de diagnóstico en paralelo.

Los tres paquetes de esta actualización comparten, una vez más, el mismo patrón de trabajo que atraviesa todo este review: un caso de uso real señalado por quien usa la herramienta en el día a día, una investigación hasta la causa raíz exacta — no un parche superficial — y una entrada de changelog que documenta tanto la solución elegida como las alternativas descartadas y por qué. Cinco entregas más, y la conclusión de este artículo sigue siendo la misma que al principio: el activo más valioso de CableDoc no es ninguna función puntual, es la disciplina con la que cada función nueva se integra al resto del sistema sin duplicar lo que ya existe.

CableDoc Desktop es, en el sentido más literal de la palabra, un proyecto de ingeniería de dominio: no es un CRUD con forma de inventario de cables, es un modelo topológico que sabe razonar sobre sí mismo. La decisión de apoyarse en un motor de grafos real (GraphQLite) en vez de reimplementar recorridos a mano, la separación disciplinada entre catálogo e instancia replicada consistentemente en cada subsistema nuevo, y — sobre todo — la transparencia de la bitácora de desarrollo, donde cada bug de producción está documentado con su causa raíz, su fix y su validación contra datos reales, son las señales que yo buscaría si tuviera que evaluar si vale la pena construir sobre esta base para los próximos años.

Los puntos de fricción son reales pero manejables: dos archivos que concentran demasiada superficie, una vista con contrato posicional en vez de nombrado, y una cobertura de pruebas todavía atada a la disponibilidad de un entorno gráfico. Ninguno de los tres es un defecto de diseño fundamental — son, más bien, el tipo de deuda que cualquier proyecto activo y de un solo desarrollador acumula cuando prioriza — correctamente, a mi juicio — entregar valor de dominio real (un asistente de diagnóstico por bisección, un modo de simulación de contingencia, un índice de riesgo auditable) por sobre pulir arquitectura que ya funciona.

Si algo distingue a CableDoc de la enorme mayoría de herramientas de documentación de planta que he visto en veinte años de carrera — muchas de ellas hojas de cálculo glorificadas, o software comercial que trata al cableado

como texto libre — es que trata la topología física como lo que es: un grafo. Y una vez que se toma esa decisión en serio, todo lo demás — impacto, riesgo, diagnóstico, contingencia, sincronización entre puestos de trabajo, e incluso una versión completa para consultar desde el celular en medio del rack — deja de ser una feature aparte y pasa a ser una consulta más, o una capa de presentación más, sobre el mismo modelo.

Ficha del redactor

Ingeniero con recorrido en mantenimiento de plantas de broadcast (racks, patcheras, cadenas de transmisión) y paso posterior por desarrollo de software de gestión de infraestructura. Escribe sobre la intersección entre ingeniería de dominio y arquitectura de software para publicaciones especializadas del sector. Este review fue encargado como auditoría técnica independiente, sin relación comercial con el desarrollo de CableDoc Desktop.

Nota metodológica: este review se realizó sobre el código fuente completo del proyecto tal como fue provisto, incluyendo el historial de `changelog.txt`, el esquema SQL (`schema_db.sql`), el `README.md` y la carpeta de planes de desarrollo (`plans/`). Las cifras de líneas de código, tablas, vistas y claves de traducción fueron medidas directamente sobre los archivos fuente, no estimadas. Los conteos operativos citados (equipos impactados, patcheras con nombreo atípico, tiempos de carga antes/después de una optimización) provienen de las entradas correspondientes del changelog del proyecto y no fueron re-verificados de forma independiente contra la base de datos de producción.